Source Code Listing
CodeAlpha Email Extractor

CodeAlpha Python Programming Internship — Task 3 (2 of 2 tasks completed)

SUMMARY

• Full source of extract_emails.py, sample_document.txt, test_extract_emails.py, and EmailExtractor_Demo.html.
• Companion document: CodeAlpha_PythonProgramming_ResearchPaper.pdf.

Author: Mayank Swaraj (Mayur Krishna)
Repository naming: CodeAlpha_EmailExtractor

Date: July 2026

Source File: extract_emails.py

```
#!/usr/bin/env python3
"""
CodeAlpha Python Programming Task 3 — Task Automation
Extracts all email addresses from a .txt file and saves them to another file.

Automates a genuinely repetitive task: manually scanning a document for email
addresses is tedious and error-prone; this script does it reliably via a
regular expression, deduplicates results, and reports what it found.
"""

import argparse
import re
import sys

# A practical (not RFC-5322-complete) email regex: matches the vast majority
# of real-world addresses while staying readable. Case-insensitive.
EMAIL_PATTERN = re.compile(
    r'[a-zA-Z0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9][a-zA-Z0-9.-]*\.[a-zA-Z]{2,}'
)


def extract_emails(text):
    """Returns a sorted list of unique email addresses found in `text`."""
    found = EMAIL_PATTERN.findall(text)
    # Normalize case for deduplication (email domains are case-insensitive;
    # local parts technically aren't, but in practice this is the expected behavior).
    seen = {}
    for email in found:
        key = email.lower()
        if key not in seen:
            seen[key] = email
    return sorted(seen.values(), key=str.lower)


def extract_from_file(input_path):
    with open(input_path, "r", encoding="utf-8", errors="replace") as f:
        text = f.read()
    return extract_emails(text)


def save_emails(emails, output_path):
    with open(output_path, "w", encoding="utf-8") as f:
        for email in emails:
            f.write(email + "\n")


def main():
    parser = argparse.ArgumentParser(description="Extract email addresses from a text file.")
    parser.add_argument("input_file", help="Path to the input .txt file to scan")
    parser.add_argument("-o", "--output", default="extracted_emails.txt",
                        help="Path to write the extracted emails (default:
                          extracted_emails.txt)")
    args = parser.parse_args()

    try:
        emails = extract_from_file(args.input_file)
    except FileNotFoundError:
        print(f"Error: input file '{args.input_file}' not found.", file=sys.stderr)
        sys.exit(1)

    save_emails(emails, args.output)
    print(f"Found {len(emails)} unique email address(es) in '{args.input_file}'.")
    print(f"Saved to '{args.output}'.")
    for e in emails:
        print(f"  {e}")


if __name__ == "__main__":
    main()
```

Source File: sample_document.txt

```
Meeting Notes — Project Kickoff, July 2026

Attendees: please confirm your availability by replying to sarah.jones@company.com or
cc'ing the project lead at m.chen99@company.co.uk before Friday.

For technical questions, reach out to dev-support@example.io (response time ~24hrs).
Do NOT use the old address, tech.support@example.com, which was decommissioned last month.

External vendor contact: procurement@vendor-solutions.net, alt contact
  billing@vendor-solutions.net
(same address — should only be counted once).

Random noise in this doc that should NOT match:
- A version number: v2.3@build (not a real email, no valid domain after @)
- A twitter-style mention: @sarah_jones (no @domain.tld structure)
- Not.an.email (missing @ entirely)
- almost@valid (no dot after the domain part)

CASE VARIANT TEST: Sarah.Jones@Company.com (same as sarah.jones@company.com above,
different casing — should be treated as the same address and only appear once in output).

Final contact for urgent issues: urgent+oncall@company.com
```

Source File: test_extract_emails.py

```python
#!/usr/bin/env python3
"""Automated tests for extract_emails.py, run against the real sample_document.txt
and additional targeted edge-case strings."""

from extract_emails import extract_emails, extract_from_file


def test_real_sample_document():
    emails = extract_from_file("sample_document.txt")
    expected = sorted([
        "sarah.jones@company.com", "m.chen99@company.co.uk", "dev-support@example.io",
        "tech.support@example.com", "procurement@vendor-solutions.net",
        "billing@vendor-solutions.net", "urgent+oncall@company.com",
    ], key=str.lower)
    assert emails == expected, f"Expected {expected}, got {emails}"
    print(f"test_real_sample_document: PASS ({len(emails)} emails, deduped correctly)")


def test_case_insensitive_deduplication():
    text = "Contact Sarah.Jones@Company.com or sarah.jones@company.com for details."
    emails = extract_emails(text)
    assert len(emails) == 1, f"Expected 1 deduplicated email, got {len(emails)}: {emails}"
    print("test_case_insensitive_deduplication: PASS")


def test_rejects_non_emails():
    text = "Not an email: @mention, v2.3@build, almost@valid, missing.at.sign, ok@ok"
    emails = extract_emails(text)
    # 'ok@ok' has no dot in the domain, so it should NOT match; nothing here should match.
    assert emails == [], f"Expected no matches, got {emails}"
    print("test_rejects_non_emails: PASS")


def test_extracts_plus_and_hyphen_addresses():
    text = "Reach us at urgent+oncall@my-company.io for emergencies."
    emails = extract_emails(text)
    assert emails == ["urgent+oncall@my-company.io"], f"Got {emails}"
    print("test_extracts_plus_and_hyphen_addresses: PASS")


def test_multiple_emails_same_line():
    text = "Emails: a@x.com, b@y.com, c@z.com all on one line."
    emails = extract_emails(text)
    assert emails == ["a@x.com", "b@y.com", "c@z.com"], f"Got {emails}"
    print("test_multiple_emails_same_line: PASS")


def test_empty_text():
    assert extract_emails("") == []
```

```
    assert extract_emails("no emails here at all") == []
    print("test_empty_text: PASS")


if __name__ == "__main__":
    test_real_sample_document()
    test_case_insensitive_deduplication()
    test_rejects_non_emails()
    test_extracts_plus_and_hyphen_addresses()
    test_multiple_emails_same_line()
    test_empty_text()
    print("\nALL TESTS PASSED")

```

Source File: EmailExtractor_Demo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=1.0">
<title>EmailExtractor — CodeAlpha Python Task 3</title>
<style>
  :root{
    --bg:#0F1419; --panel:#181D24; --border:#262D36; --accent:#5AA9E6; --good:#4ED08A;
    --text:#E7ECF2; --muted:#7C8899;
  }
  *{box-sizing:border-box;}
  html,body{margin:0;padding:0;background:var(--bg);}
  body{font-family:'Inter',system-ui,-apple-system,sans-serif;color:var(--text);display:flex;justi
    fy-content:center;}
  #app{width:100%;max-width:480px;min-height:100vh;padding-bottom:30px;}
  header{padding:20px 18px 12px;}
  .brand{font-family:'Space Grotesk','Inter',sans-serif;font-weight:700;font-size:19px;}
  .brand span{color:var(--accent);}
  .sub{font-size:11px;color:var(--muted);margin-top:4px;}

  #form{padding:0 18px 12px;}
  textarea{
    width:100%;min-height:160px;padding:12px 14px;border-radius:11px;border:1px solid
      var(--border);
    background:var(--panel);color:var(--text);font-size:12.5px;font-family:'JetBrains
      Mono',monospace;
    resize:vertical;margin-bottom:8px;line-height:1.6;
  }
  button{width:100%;padding:12px;border-radius:10px;border:none;background:var(--accent);color:#04
    141F;
    font-weight:700;font-size:14px;cursor:pointer;}

  #presets{display:flex;gap:6px;padding:0 18px 14px;}
  .preset-btn{flex:1;font-size:10.5px;font-weight:600;padding:8px 4px;border-radius:8px;border:1px
    solid var(--border);background:var(--panel);color:var(--muted);cursor:pointer;}

  #resultCard{margin:0 18px
    20px;padding:14px;border-radius:14px;background:var(--panel);border:1px solid var(--border);}
  .result-title{font-size:11px;color:var(--muted);text-transform:uppercase;letter-spacing:.04em;ma
    rgin-bottom:8px;font-weight:700;}
  .email-chip{
    font-family:'JetBrains Mono',monospace;font-size:11.5px;background:var(--bg);border:1px solid
      var(--border);
    border-radius:7px;padding:7px 10px;margin-bottom:5px;color:var(--good);
  }
  .empty{color:var(--muted);font-size:12px;text-align:center;padding:10px 0;}

  #note{padding:0 18px 30px;font-size:10.5px;color:var(--muted);line-height:1.5;}
</style>
</head>
<body>
<div id="app">
  <header>
    <div class="brand">Email<span>Extractor</span></div>
    <div class="sub">CodeAlpha Python Task 3 — Task Automation (extract emails from text)</div>
  </header>

  <div id="form">
```

```
    <textarea id="textInput" placeholder="Paste text containing email addresses..."></textarea>
    <button id="extractBtn">Extract Email Addresses</button>
  </div>

  <div id="presets">
    <button class="preset-btn" id="loadSampleBtn">Load real sample document</button>
    <button class="preset-btn" id="clearBtn">Clear</button>
  </div>

  <div id="resultCard">
    <div class="result-title" id="resultTitle">No emails extracted yet</div>
    <div id="emailList"></div>
  </div>

  <div id="note">This mirrors extract_emails.py's exact regex
    ([a-zA-Z0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9]
    [a-zA-Z0-9.-]*\.[a-zA-Z]{2,}) and case-insensitive deduplication logic in JavaScript. The
      "Load
    real sample document" button loads the exact sample_document.txt used to verify the Python
      script,
    including deliberately tricky non-email text (version numbers, @mentions) that should NOT
      match.
    The real Python script was verified with 6 automated tests — see the research paper and source
    code PDF.</div>
</div>

<script>
/* ============================================================
   CodeAlpha Python Task 3 — Email Extractor (JS mirror of extract_emails.py)
   ============================================================ */

const EMAIL_PATTERN = /[a-zA-Z0-9][a-zA-Z0-9._%+-]*@[a-zA-Z0-9][a-zA-Z0-9.-]*\.[a-zA-Z]{2,}/g;

function extractEmails(text){
  const found = text.match(EMAIL_PATTERN) || [];
  const seen = new Map();
  found.forEach(email => {
    const key = email.toLowerCase();
    if(!seen.has(key)) seen.set(key, email);
  });
  return Array.from(seen.values()).sort((a,b) => a.toLowerCase().localeCompare(b.toLowerCase()));
}

const SAMPLE_DOCUMENT = `Meeting Notes — Project Kickoff, July 2026

Attendees: please confirm your availability by replying to sarah.jones@company.com or
cc'ing the project lead at m.chen99@company.co.uk before Friday.

For technical questions, reach out to dev-support@example.io (response time ~24hrs).
Do NOT use the old address, tech.support@example.com, which was decommissioned last month.

External vendor contact: procurement@vendor-solutions.net, alt contact
  billing@vendor-solutions.net
(same address — should only be counted once).

Random noise in this doc that should NOT match:
- A version number: v2.3@build (not a real email, no valid domain after @)
- A twitter-style mention: @sarah_jones (no @domain.tld structure)
- Not.an.email (missing @ entirely)
- almost@valid (no dot after the domain part)

CASE VARIANT TEST: Sarah.Jones@Company.com (same as sarah.jones@company.com above,
different casing — should be treated as the same address and only appear once in output).

Final contact for urgent issues: urgent+oncall@company.com`;

function renderResults(emails){
  const titleEl = document.getElementById('resultTitle');
  const listEl = document.getElementById('emailList');
  titleEl.textContent = emails.length > 0
    ? `Found ${emails.length} unique email address${emails.length===1?'':'es'}`
    : 'No emails found';
  listEl.innerHTML = emails.length > 0
    ? emails.map(e => `<div class="email-chip">${e}</div>`).join('')
    : '<div class="empty">Paste some text and click Extract.</div>';
}

document.getElementById('extractBtn').addEventListener('click', () => {
```

```
138 |   const text = document.getElementById('textInput').value;
139 |   renderResults(extractEmails(text));
140 | });
141 |
142 | document.getElementById('loadSampleBtn').addEventListener('click', () => {
143 |   document.getElementById('textInput').value = SAMPLE_DOCUMENT;
144 |   renderResults(extractEmails(SAMPLE_DOCUMENT));
145 | });
146 |
147 | document.getElementById('clearBtn').addEventListener('click', () => {
148 |   document.getElementById('textInput').value = '';
149 |   renderResults([]);
150 | });
151 |
152 | renderResults([]);
153 | </script>
154 | </body>
155 | </html>
156 |
```